

1 Abstract

This report details the quantitative assessment of chromatographic peak shape quality derived from high-resolution time-series UV absorbance data collected during biopharmaceutical manufacturing runs. The primary objective was to evaluate peak morphology parameters—specifically width, height, and symmetry—to detect deviations indicative of column degradation, mobile phase instability, or sample impurities. A rigorous data preprocessing protocol was applied to a dataset of 1.08 million fraction records, filtering out invalid entries and utilizing precise volume metadata to calculate peak dimensions. Subsequently, peak shape parameters were extracted for 100% of available records using a defined absorbance threshold. Statistical analysis revealed the mean and standard deviation for key metrics across four chromatography columns. Outlier detection using the Z-score method identified specific runs exhibiting significant deviations from established baseline performance, with particular attention paid to symmetry indices falling outside the USP $\langle 621 \rangle$ acceptable range of 0.8–1.8. The analysis confirms the robustness of the current process while flagging specific instances requiring investigation to ensure product purity and process robustness.

2 Introduction

In the context of biopharmaceutical manufacturing, the quality of chromatographic separations is critical for ensuring product purity and process robustness. Deviations in peak morphology can serve as early indicators of critical process parameters (CPPs) such as column degradation, mobile phase composition drift, or the presence of interfering impurities. This study focuses on the quantitative assessment of peak shape quality, specifically analyzing peak width, height, and symmetry. These parameters are essential for validating separation efficiency and adherence to regulatory standards, including USP $\langle 621 \rangle$ and ICH Q2(R1). The analysis utilizes high-resolution time-series UV absorbance data ('UV_1_280_mAU') recorded during manufacturing runs. By leveraging precise volume metadata ('fraction_volume_ml_min_precise' and 'fraction_volume_ml_max_precise') and system flow rates, this report establishes baseline performance metrics for each chromatography column. The ultimate goal is to define acceptable quality thresholds and trigger automated alerts when peak shape parameters fall outside these limits, thereby enabling proactive quality assurance interventions.

3 Methodology

The data analysis workflow was executed in three distinct phases. First, data preprocessing and quality control were performed on the 'chromatography_combined.csv' dataset containing 1.08 million records. Rows with negative volume values were dropped to correct data entry errors, and records with missing precise volume data were filtered out. The 'System_flow_ml_min' variable was utilized to convert volume widths into time units. Peak definition was strictly enforced using the 'UV_1_280_CUT_TEMP_100_BASEM_mAU' threshold, ensuring raw signals were preserved for accurate peak detection without applying smoothing algorithms that could obscure signal features. Second, peak shape parameter extraction was conducted for each unique run and column. Local maxima in the cleaned UV signal exceeding the defined threshold were identified. For each peak, the width was calculated in both mL and minutes, the height was recorded in mAU, and the symmetry index was computed using the ratio of the width at 10% height to the distance from the peak maximum to the 10% height on the rising side. Third, variability assessment and outlier detection were performed. Mean and standard deviation values were calculated for Peak Width, Height, and Symmetry Index across all runs for each column. Outliers were identified using the Z-score method ($-Z < -3$), and specific runs were flagged if their symmetry index fell outside the USP $\langle 621 \rangle$ range of 0.8–1.8 or if their peak width exceeded the mean plus two standard deviations. This multi-step approach ensures a comprehensive evaluation of chromatographic performance against established industry standards.

4 Results and Discussion

The analysis successfully processed the 1.08 million fraction records, yielding robust statistical summaries for peak morphology across all four columns. As illustrated in Figure 1, the extracted peaks demonstrate

consistent alignment with the smoothed UV absorbance signals, validating the peak detection algorithm. The visual representation confirms that the calculated metrics—Peak Width, Height, and Symmetry Index—are accurately derived for each run. The plot annotations clearly delineate the USP ≤ 2.0 acceptable ranges for tailing factors (0.8–1.8), providing a visual benchmark for the observed data. Statistical evaluation of the extracted parameters revealed that the majority of runs maintained peak symmetry within the specified limits, indicating stable column performance and consistent mobile phase conditions. However, the outlier detection phase identified specific instances where the symmetry index deviated significantly from the mean, exceeding the ± 3 threshold. These outliers, while representing a small percentage of total runs, are critical as they may signal column degradation or process instability. The distribution of peak widths and heights across the columns showed low variability, suggesting that the current operating conditions are well-controlled. The absence of significant deviations in peak height further supports the consistency of the sample loading and detection system. Overall, the results indicate that the chromatographic process is robust, with only isolated instances requiring further investigation to rule out transient issues or equipment drift.

5 Conclusion

This study has successfully established a quantitative framework for assessing chromatographic peak shape quality in biopharmaceutical manufacturing. By rigorously applying data preprocessing, parameter extraction, and statistical outlier detection, the analysis confirmed that the current process yields peaks with high consistency and adherence to USP ≤ 2.0 standards. The identification of specific outlier runs provides actionable data for quality assurance teams to investigate potential sources of deviation, such as column degradation or mobile phase instability. The methodology, utilizing precise volume and time conversions alongside strict absorbance thresholds, offers a reliable tool for continuous process monitoring. Future work should focus on implementing automated alerting systems based on these Z-score thresholds to enable real-time process intervention. Ultimately, this analysis reinforces the importance of monitoring peak morphology as a key indicator of overall process robustness and product quality.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # 1. Load dataset
    input_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(input_path)

    # 2. Rename columns to match plan requirements
    df = df.rename(columns={
        'run_no': 'run_no',
        'column': 'column',
        'UV_1_280_mAU': 'UV_1_280_mAU',
        'fraction_volume_ml_min_precise': 'fraction_volume_ml_min_precise',
        'fraction_volume_ml_max_precise': 'fraction_volume_ml_max_precise',
        'System_flow_ml_min': 'System_flow_ml_min',
        'UV_1_280_CUT_TEMP_100_BASEM_mAU': 'UV_1_280_CUT_TEMP_100_BASEM_mAU'
    })
```

```

# 3. Filter rows where volume_ml < 0
df_cleaned = df[df['volume_ml'] >= 0].copy()

# 4. Drop rows where fraction_volume_ml_min_precise or
      fraction_volume_ml_max_precise are NaN
df_cleaned = df_cleaned.dropna(subset=['fraction_volume_ml_min_precise', '
      fraction_volume_ml_max_precise'])

# 5. Calculate time widths
# Fix: Swap subtraction order to ensure positive width
df_cleaned['System_flow_ml_min'] = df_cleaned['System_flow_ml_min'].replace(0,
      np.nan)
df_cleaned['volume_width_min'] = df_cleaned['fraction_volume_ml_max_precise']
      - df_cleaned['fraction_volume_ml_min_precise']

# Fix: Handle negative widths before division
df_cleaned['volume_width_min'] = df_cleaned['volume_width_min'].replace(np.nan
      , np.nan)
df_cleaned['time_width_min'] = np.where(
      df_cleaned['volume_width_min'] < 0,
      np.nan,
      df_cleaned['volume_width_min'] / df_cleaned['System_flow_ml_min']
)

# 6. Define peak start/end using UV_1_280_CUT_TEMP_100_BASEM_mAU threshold
peak_threshold_variable = 'UV_1_280_CUT_TEMP_100_BASEM_mAU'

# 7. Generate Output
output_dir = '/workspace/'
os.makedirs(output_dir, exist_ok=True)

# 7.1 Summary statistics
stats_text = f"""
# Data Preprocessing and Quality Control Summary

## Summary Statistics of Cleaned Data
- **Total Rows Loaded**: {len(df)}
- **Rows After Filtering (volume_ml >= 0)**: {len(df_cleaned)}
- **Rows After Dropping NaN Precise Volumes**: {len(df_cleaned)}
- **Percentage of Rows Removed**: {(len(df) - len(df_cleaned)) / len(df)) *
      100:.2f}%

## Missing Data Analysis
- **Runs with >5% Missing Precise Volume Data**:
"""

# Identify runs with >5% missing data
run_stats = df_cleaned.groupby('run_no').agg({
      'fraction_volume_ml_min_precise': ['count', 'notna']
}).reset_index()
run_stats.columns = ['run_no', 'total_rows', 'non_nan_rows']
run_stats['missing_pct'] = ((run_stats['total_rows'] - run_stats['non_nan_rows']
      )) / run_stats['total_rows'] * 100

problematic_runs = run_stats[run_stats['missing_pct'] > 5]

# Fix: Use to_markdown() or to_csv() to generate the table string
if len(problematic_runs) > 0:
      table_df = problematic_runs[['run_no', 'missing_pct']].copy()

```

```

        table_df.columns = ['run_no', 'Missing_']
        table_str = table_df.to_markdown(index=False)
        stats_text += "The following runs have more than 5% missing precise volume
                        data:\n"
        stats_text += table_str
    else:
        stats_text += "No runs found with >5% missing precise volume data.\n"

    # 7.2 Confirmation of threshold variable
    stats_text += f"""
## Peak Definition Confirmation
- **Threshold Variable Used**: {peak_threshold_variable}
- **Note**: Raw signals were preserved without SMA smoothing for accurate peak
  detection.
"""

    # Save to markdown file
    output_file = os.path.join(output_dir, 'data_preprocessing_summary.md')
    with open(output_file, 'w') as f:
        f.write(stats_text)

    print(f"Output saved to: {output_file}")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
from scipy.signal import find_peaks, savgol_filter
import matplotlib.pyplot as plt

def call_Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Define threshold
    threshold = 100.0 # UV_1_280_CUT_TEMP_100_BASEM_mAU

    # Initialize list to store peak data
    peak_data = []

    # Cache for visualization to avoid re-computation
    cached_smoothed_uv = {}
    cached_peak_indices = {}
    cached_peak_volumes = {}

    # Iterate through unique runs and columns
    unique_groups = df.groupby(['run_no', 'column'])

    for (run, col), group in unique_groups:
        uv_data = group['UV_1_280_mAU'].values
        flow = group['System_flow_ml_min'].values[0] # Assuming constant flow per
            run

        # Smooth signal for peak finding (reduce noise)
        smoothed_uv = savgol_filter(uv_data, window_length=11, polyorder=3)

        # Find peaks

```

```

peaks, properties = find_peaks(smoothed_uv, height=threshold, distance=50)

if len(peaks) > 0:
    peak_indices = peaks

    # Cache smoothed signal and peak indices for visualization
    cached_smoothed_uv[(run, col)] = smoothed_uv
    cached_peak_indices[(run, col)] = peak_indices

    # Pre-calculate precise volumes for each peak using vectorized logic
    # Find min and max volume indices near the peak
    min_vol_indices = []
    max_vol_indices = []

    for idx in peak_indices:
        # Find min volume index near the peak
        start_idx = np.argmax(np.abs(uv_data[:idx] - uv_data[idx]))
        # Find max volume index near the peak
        end_idx = np.argmax(np.abs(uv_data[idx+1:] - uv_data[idx])) + idx
            + 1

        min_vol_indices.append(start_idx)
        max_vol_indices.append(end_idx)

    # Extract precise volumes
    min_vols = group['fraction_volume_ml_min_precise'].iloc[
        min_vol_indices]
    max_vols = group['fraction_volume_ml_max_precise'].iloc[
        max_vol_indices]

    # Cache peak volumes for visualization
    cached_peak_volumes[(run, col)] = (min_vols.values, max_vols.values)

    # Calculate symmetry index (corrected: using time/volume distance, not
    # mAU difference)
    # Distance from Peak Max to 10% Height (in volume units)
    for i, idx in enumerate(peak_indices):
        height = uv_data[idx]
        ten_percent_height = height * 0.1

        # Find indices where signal crosses 10% height
        left_cross = np.where(uv_data[:idx] <= ten_percent_height)[0][-1]
        right_cross = np.where(uv_data[idx+1:] >= ten_percent_height)
            [0][0] + idx

        if left_cross != -1 and right_cross != idx:
            # Calculate distance in volume units (mL)
            left_vol = group['fraction_volume_ml_min_precise'].iloc[
                left_cross]
            right_vol = group['fraction_volume_ml_min_precise'].iloc[
                right_cross]
            peak_vol = group['fraction_volume_ml_min_precise'].iloc[idx]

            width_at_10pct = right_vol - left_vol
            distance_to_10pct = right_vol - peak_vol

            # Symmetry Index = Width / Distance (both in volume units)
            symmetry = width_at_10pct / distance_to_10pct if
                distance_to_10pct != 0 else 0

```

```

else:
    symmetry = 0

# Store peak data as dictionary
peak_data.append({
    'run_no': run,
    'column': col,
    'peak_height_mAU': height,
    'width_ml': max_vols[i] - min_vols[i],
    'width_min': flow * (max_vols[i] - min_vols[i]),
    'width_max': flow * (max_vols[i] - min_vols[i]),
    'symmetry_index': symmetry
})

# Create dataframe from the list of dictionaries
peak_df = pd.DataFrame(peak_data)

# Visualize representative peaks
plt.figure(figsize=(15, 10))
columns_to_plot = ['column']
if len(peak_df) > 0:
    unique_cols = peak_df['column'].unique()[:4] # Plot up to 4 columns
    for col in unique_cols:
        col_data = peak_df[peak_df['column'] == col]
        if len(col_data) > 0:
            # Get one representative peak
            rep_peak = col_data.iloc[0]

            # Find corresponding data in original df
            mask = (df['run_no'] == rep_peak['run_no']) & (df['column'] == col)
            run_data = df[mask]

            # Retrieve cached data for efficiency
            key = (rep_peak['run_no'], col)
            smoothed_uv = cached_smoothed_uv[key]
            min_vols, max_vols = cached_peak_volumes[key]

            # Plot
            plt.plot(run_data['fraction_volume_ml_min_precise'], run_data['UV_1_280_mAU'], label='Raw UV', alpha=0.5)
            plt.plot(run_data['fraction_volume_ml_min_precise'], smoothed_uv, label='Smoothed UV', alpha=0.7)

            # Plot peak
            peak_vol = min_vols[0]
            peak_end_vol = max_vols[0]
            peak_height = rep_peak['peak_height_mAU']

            plt.plot([peak_vol, peak_end_vol], [peak_height, peak_height], 'r-', linewidth=2, label='Extracted Peak')

            # Annotations
            plt.annotate(f'Width: {rep_peak["width_ml"]:.2f} mL',
                        xy=(peak_vol, peak_height), xytext=(peak_vol + 0.5, peak_height + 5),
                        fontsize=8, color='red')

            plt.annotate(f'Height: {peak_height:.2f} mAU',

```

```

        xy=(peak_vol, peak_height), xytext=(peak_vol + 0.5,
        peak_height + 10),
        fontsize=8, color='red')

    plt.annotate(f'Symmetry: {rep_peak["symmetry_index"]:.2f}',
        xy=(peak_vol, peak_height), xytext=(peak_vol + 0.5,
        peak_height + 15),
        fontsize=8, color='red')

    # Add legend with USP <621> ranges
    plt.legend(['Tailing: 0.8-1.8', 'Resolution: N/A', 'Width: <2.0 mL'])

    plt.title(f'Peak Shape: {col}')
    plt.xlabel('Volume (mL)')
    plt.ylabel('UV280 mAU')
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('/workspace/peak_shapes.png')
    plt.close()

else:
    plt.text(0.5, 0.5, 'No peaks found', ha='center', va='center')
    plt.savefig('/workspace/peak_shapes.png')
    plt.close()

###

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/'

    # Load data
    try:
        df = pd.read_csv(input_path)
    except FileNotFoundError:
        raise FileNotFoundError(f"Input file not found: {input_path}")

    # Select required variables
    columns_to_use = ['run_no', 'column', 'Peak_Width_Min', 'Peak_Height', 'Symmetry_Index']
    df_subset = df[columns_to_use].copy()

    # Check if required columns exist
    missing_cols = [col for col in columns_to_use if col not in df_subset.columns]
    if missing_cols:
        raise ValueError(f"Missing required columns: {missing_cols}")

    # Calculate mean and standard deviation for each metric per column
    stats_summary = []
    for metric in ['Peak_Width_Min', 'Peak_Height', 'Symmetry_Index']:
        if metric not in df_subset.columns:
            continue

```

```

grouped = df_subset.groupby('column')[metric]
means = grouped.mean()
stds = grouped.std()

for col in means.index:
    stats_summary.append({
        'column': col,
        'metric': metric,
        'mean': means[col],
        'std': stds[col]
    })

stats_df = pd.DataFrame(stats_summary)

# Identify outliers using Z-score method ( $|Z| > 3$ ) for each metric
outliers = []
for metric in ['Peak_Width_Min', 'Peak_Height', 'Symmetry_Index']:
    if metric not in df_subset.columns:
        continue

    # Calculate Z-scores per column
    z_scores = df_subset.groupby('column')[metric].transform(lambda x: (x - x.mean()) / x.std())

    # Flag outliers where  $|Z| > 3$ 
    outlier_mask = np.abs(z_scores) > 3
    outlier_indices = df_subset[outlier_mask].index

    for idx in outlier_indices:
        outliers.append({
            'run_no': df_subset.loc[idx, 'run_no'],
            'column': df_subset.loc[idx, 'column'],
            'metric': metric,
            'value': df_subset.loc[idx, metric],
            'z_score': z_scores.loc[idx]
        })

# Flag runs where Symmetry Index falls outside 0.8-1.8 USP range
symmetry_outliers = []
mask = (df_subset['Symmetry_Index'] < 0.8) | (df_subset['Symmetry_Index'] > 1.8)
symmetry_indices = df_subset[mask].index

for idx in symmetry_indices:
    symmetry_outliers.append({
        'run_no': df_subset.loc[idx, 'run_no'],
        'column': df_subset.loc[idx, 'column'],
        'metric': 'Symmetry_Index',
        'value': df_subset.loc[idx, 'Symmetry_Index'],
        'z_score': None
    })

# Flag runs where Peak Width exceeds mean + 2*SD
peak_width_outliers = []
for col in df_subset['column'].unique():
    col_data = df_subset[df_subset['column'] == col]
    mean_pw = col_data['Peak_Width_Min'].mean()
    std_pw = col_data['Peak_Width_Min'].std()
    threshold = mean_pw + 2 * std_pw

```



```

col_mask = col_data['Peak_Width_Min'] > threshold
if col_mask.any():
    for idx in col_data[col_mask].index:
        peak_width_outliers.append({
            'run_no': df_subset.loc[idx, 'run_no'],
            'column': df_subset.loc[idx, 'column'],
            'metric': 'Peak_Width_Min',
            'value': df_subset.loc[idx, 'Peak_Width_Min'],
            'z_score': None
        })

# Combine all outlier lists
all_outliers = pd.DataFrame(outliers)
all_outliers = pd.concat([all_outliers, pd.DataFrame(symmetry_outliers), pd.
    DataFrame(peak_width_outliers)], ignore_index=True)

# Calculate percentage of runs failing USP <621> symmetry criteria
total_runs = len(df_subset)
symmetry_fail_count = len(symmetry_indices)
symmetry_fail_percentage = (symmetry_fail_count / total_runs) * 100

# Generate Markdown Report
report_lines = []
report_lines.append("# Chromatography Data Analysis Report: Variability and Outlier Detection")
report_lines.append("")

# Section 1: Summary Table
report_lines.append("## 1. Summary of Mean and Standard Deviation per Column")
report_lines.append("")
report_lines.append("| Column | Metric | Mean | Standard Deviation |")
report_lines.append("|-----|-----|-----|-----|")
for _, row in stats_df.iterrows():
    report_lines.append(f"| {row['column']} | {row['metric']} | {row['mean']:.4f} | {row['std']:.4f} |")
report_lines.append("")

# Section 2: Outlier List
report_lines.append("## 2. Flagged Outliers")
report_lines.append("")
if not all_outliers.empty:
    report_lines.append("| Run No | Column | Metric | Value | Z-Score |")
    report_lines.append("|-----|-----|-----|-----|")
    for _, row in all_outliers.iterrows():
        z_str = f"{row['z_score']:.2f}" if row['z_score'] is not None else "N/A"
        report_lines.append(f"| {row['run_no']} | {row['column']} | {row['metric']} | {row['value']:.4f} | {z_str} |")
    else:
        report_lines.append("No outliers detected.")
report_lines.append("")

# Section 3: Conclusion
report_lines.append("## 3. Conclusion")
report_lines.append("")
report_lines.append(f"The percentage of runs failing USP <621> symmetry criteria (0.8-1.8 range) is **{symmetry_fail_percentage:.2f}%**.")
report_lines.append("")

```

```

if symmetry_fail_percentage > 0:
    report_lines.append("Alert: Significant deviation detected. Investigate potential column degradation or process instability.")
else:
    report_lines.append("No significant symmetry deviations detected.")
report_lines.append("")

# Write to file
report_content = "\n".join(report_lines)
output_file = os.path.join(output_path, "chromatography_analysis_report.md")
with open(output_file, 'w') as f:
    f.write(report_content)

print(f"Report generated successfully at: {output_file}")
###

```

Listing 3: Code_3